

ÖNVEZETETT LABORATÓRIUM DOKUMENTÁCIÓ

Téma: *BigData / Üzleti Intelligencia (BI) rendszer fejlesztése*

Github link

Power BI link

Tartalomjegyzék

Bevezetés	2
Miről is szól egy Big Data, BI rendszer?	2
A projekt maga: Valutaárfolyamok és hírek megfigyelése	2
Projekt szerkezete.	2
Használt technológiák	3
SQL	4
RAW és Staging táblák.....	4
FACT és DIM táblák	6
ETL jobs, SSIS.....	8
DIMdate tábla feltöltése	8
A RAW táblák feltöltése	9
A DIM táblák feltöltése	10
Az STG tábla felöltése kategóriákkal.....	11
A FACT táblák feltöltése.....	12
Power BI.....	13
View-k	13
Valuta Árfolyamok.....	13
Valuta árfolyam mátrix, hő térkép	13
Hírek, média eloszlás	15
Hírek és valuta árfolyam korreláció	15
Fejlesztési tervek.....	16

Dutka Lehel Ákos

TO0WC2

Bevezetés

Miről is szól egy Big Data, BI rendszer?

Egy modern Üzleti Intelligencia (BI) és Big Data rendszer elsődleges célja, hogy a naponta keletkező, nagy volumenű és gyakran strukturálatlan adathalmazokat befogadja, feldolgozza és értelmezhető formába öntse.

A folyamat során a rendszer kinyer akár több millió rekordot, amelyeket tisztítanak és transzformálnak, majd egy központi adattárházban eltárolnak.

Utolsó lépésként, a tárolás után pedig a feladatunk az adatok frissen tartása és a felhasználók számára való megjelenítés. Azaz olyan interaktív riportok és felületek készítése, amelyek alapján a felhasználók gyorsan és hatékonyan tudnak tájékozódni.

A projekt maga: Valutaárfolyamok és hírek megfigyelése

A projekt fókuszában egy olyan adatvezérelt architektúra megtervezése és fejlesztése áll, amely összekapcsolja a valutaárfolyamok mozgását a globális hírekkel.

A célom egy olyan platform létrehozása volt, ahol a felhasználók egyetlen felületen juthatnak releváns információkhoz a napi valutaárfolyam-ingadozásokról és az ehhez kapcsolódható hírfolyamról.

A rendszer vizuálisan is lehetővé teszi, a lehetséges ok-okozati korrelációk elemzését a napi hírek mennyisége/témája és az árfolyamok változása között.

Projekt szerkezete.

Exchange-rates_News_SSIprojekt: Az SSIS projektet tartalmazó mappa.

MSSQL: Az SQL adatbázis felépítéséhez szükséges szkriptek.

Python: A kategorizáló python szkript.

ArticleCategories.txt: A kategóriákat tartalmazó szöveges fájl, amiket kioszthat a hír cikkeknek a python szkript. Innen töltődik fel a DIMcategory tábla az ETL jobok alatt.

Használt technológiák

Sok open-source alkalmazás található egy Big Data, Business Intelligence rendszer megvalósításához, viszont én, ahol lehetséges volt a Microsoft által kiadott rendszereket használtam.

Az adatbázisért felelős programok

- **MSSQL: Microsoft SQL Server Express Edition 2025**
Microsoft ingyenes adatbázis kezelő lokális szervere, amely 50GB tárhelyet biztosít.
- **SQL server management studio 22**
Microsoft ingyenes és legfrissebb alkalmazása az adatbázis szerverek kezelésére.

Az adatok eléréséért, feldolgozásáért és frissítéséért felelős program

- **SSIS**
Microsoft vállalati szintű platformja adatintegrációs, adatmigrációs és workflow alkalmazások készítésére. Elsősorban egy ETL eszköz, amely lehetővé teszi az adatok különböző forrásokból történő kinyerését, átalakítását és tisztítását, majd betöltését például egy adattárházba.

Használt API-k

- A valutákért felelős API: [Frankfurter | Free exchange rates API](#)
Egy nyílt forráskódú és ingyenes REST API, amely az Európai Központi Bank referenciaárfolyamait szolgáltatja a devizapiaci adatok lekéréséhez.
- A hírekért felelős API: [News API – Search News and Blog Articles on the Web](#)
Egy átfogó adatszolgáltató platform, amely lehetővé teszi a weben megjelenő globális hírösszefoglalók és blogcikkek strukturált, JSON formátumban történő automatikus lekérdezését.

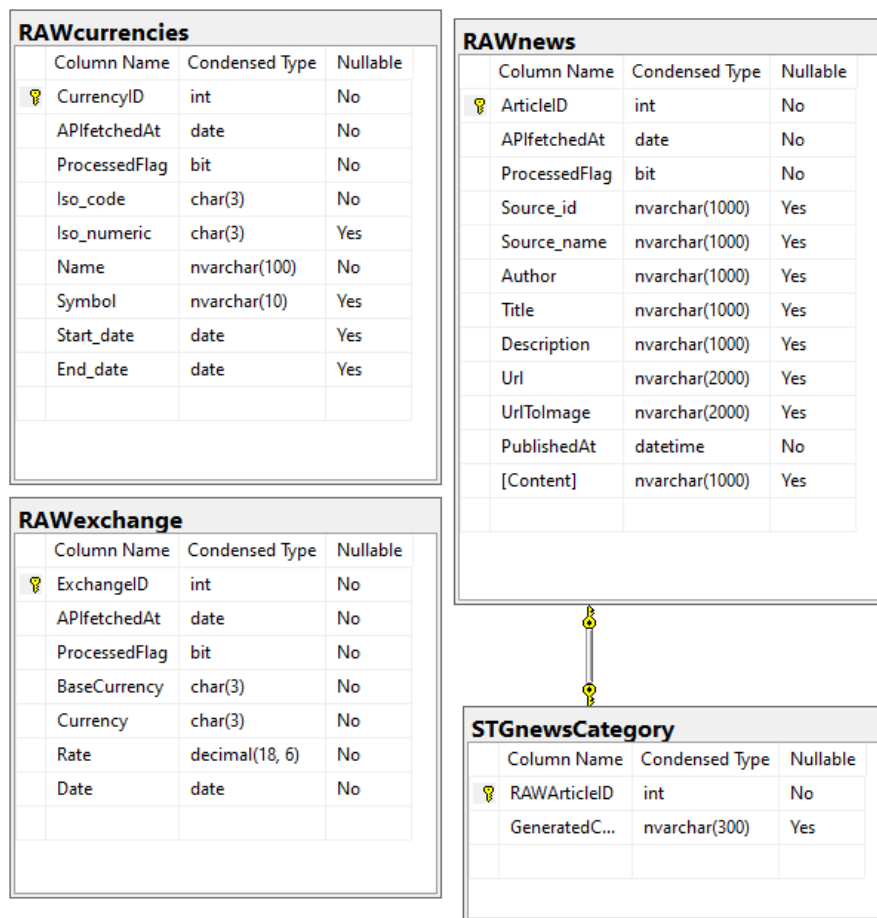
Programozási nyelvek és Mesterséges Intelligencia

- **C# és Python:** A C# szkripteket az SSIS csomagokon belül alkalmaztam az API hívások és adatkonverziók kezelésére.
A Python elsősorban a nyelvi modellel való kommunikációért felelős.
- **Ollama:** Lokálisan futtatott LLM. A projektben a hírek automatizált kategorizálását végzi. Az adatbiztonság és a költséghatékonyság érdekében egy lokálisan futtatott nyelvi modellt, **qwen2.5:1.5b**-t használtam.

Az adatok megjelenítéséért, reportok készítéséért felelős program.

- **Microsoft Power BI**
A rendszer végpontja, egy üzleti intelligencia és adatvizualizációs platform, amely több forrásból származó nyers, szétszórt adatokat alakít át használható elemzésekké, interaktív irányítópultokká és jelentésekké.

RAW és Staging táblák



A RAW táblák tárolják a nyers, módosíthatatlan adatokat, amelyeket az API hívások során kapunk. Mindegyik táblában található kettő azonos attribútum:

- **ProcessedFlag:** Ezzel az attribútummal tudjuk jelezni, hogy a rekord fel lett-e már dolgozva, azaz a FACT/DIM tábla szerkezetébe be lett-e már szűrve. Segít nekünk elkerülni felesleges rekordokkal való dolgozást, erőforrásokat takarít meg nekünk.
- **APIfetchedAt:** Esetleges naplózást segíti elő, jelzi, hogy mikori API hívás eredménye a rekord.

STGnewsCategory: Átmeneti, Staging tábla, amely a lokális AI (Ollama) által legenerált kategóriákat tárolja.

- **RAWArticleID:** Ez az attribútum egyszerre elsődleges kulcs (PK) és idegen kulcs. Egy 1:1 arányú kapcsolatot kényszerít ki, garantálva, hogy csak olyan hírhez menthessünk kategóriát, amely már létezik a RAWnews táblában.
- **GeneratedCategory:** A Python szkript és az AI modell által kiosztott, véglegesített szöveges kategória (pl. Economy, Geopolitics).

RAWcurrencies: A lekérdezett valuták adatainak nyers tárolója. Ez biztosítja, hogy a rendszer ismerje az API-ból érkező devizák alapvető tulajdonságait.

- **CurrencyID:** A rekord egyedi, automatikusan generált elsődleges kulcsa.
- **Iso_code:** A valuta nemzetközi szabvány szerinti 3 betűs azonosítója (pl. USD).
- **Iso_numeric:** A valuta numerikus ISO kódja.
- **Name:** A deviza teljes, hivatalos megnevezése.
- **Symbol:** A valuta fizikai szimbóluma (pl. \$, €).
- **Start_date / End_date:** Az API szolgáltató által biztosított dátumhatárok, amelyek jelzik a valuta adatainak elérhetőségét.

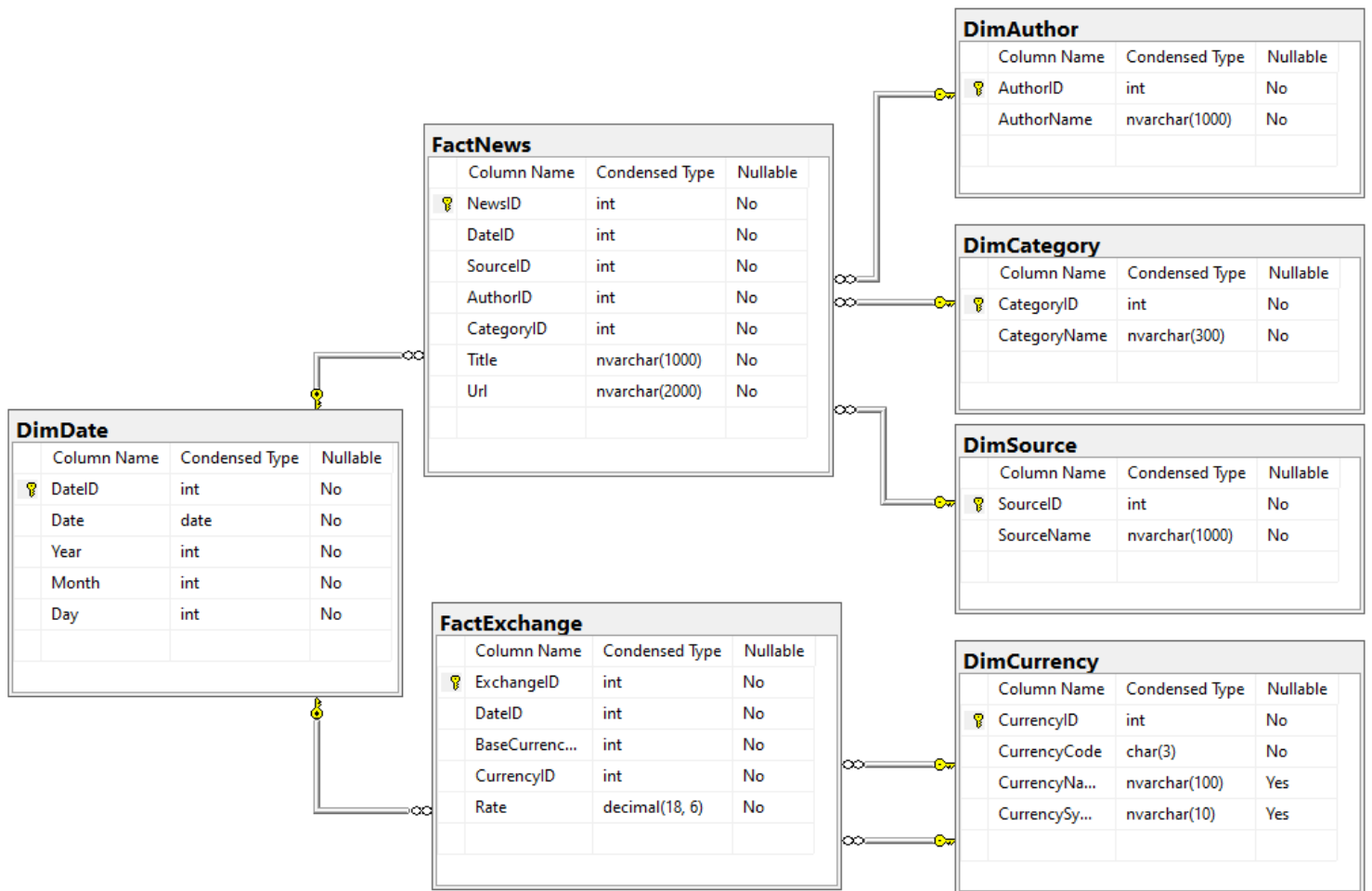
RAWexchange: A napi valutaárfolyamok tranzakciós adatainak nyers gyűjtőhelye.

- **ExchangeID:** Az árfolyamrekord egyedi, automatikusan generált elsődleges kulcsa.
- **BaseCurrency:** A viszonyítási alapként szolgáló deviza kódja.
- **Currency:** A célvaluta kódja, amelyre az átváltás történik.
- **Rate:** A tényleges átváltási árfolyam pontos, decimális értéke.
- **Date:** Az árfolyam érvényességének hivatalos dátuma.

RAWnews: A News API-ból letöltött cikkek metaadatainak és nyers szöveges tartalmának tárolója.

- **ArticleID:** A letöltött hír egyedi, automatikusan generált elsődleges kulcsa.
- **Source_id / Source_name:** A hírügynökség vagy portál azonosítója és olvasható neve (pl. Reuters, CNN).
- **Author:** A cikk írójának vagy publikálójának neve.
- **Title:** A hír főcíme.
- **Description:** A cikk rövid összefoglalója vagy bevezető szövege.
- **Url / UrlToImage:** Webes hivatkozások az eredeti cikkekre és annak borítóképére.
- **PublishedAt:** A hír publikálásának pontos, másodpercalapú időbélyege.
- **[Content]:** A cikk nyers, korlátozott hosszúságú szöveges tartalma.

FACT és DIM táblák



Adattárház.

A nyers adatok megtisztítása és transzformációja után az adatok a végleges analitikai rétegbe kerülnek, amely egy relációs **Csillagséma (Star Schema)** architektúrára épül. Ez a struktúra biztosítja a Power BI számára a leggyorsabb lekérdezési teljesítményt.

DIM (Dimenzió) Táblák

A dimenziótáblák tárolják az üzleti entitások leíró jellegű törzsadatait. Fő feladatuk, hogy kontextust, azaz szűrési és csoportosítási lehetőségeket biztosítsanak a ténytáblák számára. Ezek a táblák szigorúan duplikációmentesek, és minden rekordjuk egy egyedi helyettesítő kulccsal rendelkezik.

DimDate: A rendszer legfontosabb, **közös dimenziója**. Ez az a híd, amely összeköti a két teljesen különálló tartományt (az árfolyamokat és a híreket).

- **DatelD:** Automatikusan generált elsődleges kulcs.
- **Date, Year, Month, Day:** Az időbeli hierarchia elemei, amelyek a Power BI-ban elengedhetetlenek.

DimCurrency: A RAWcurrency-ből tisztított valuta adatokat tárolja.

- **CurrencyID:** Automatikusan generált elsődleges kulcs.
- **CurrencyCode / CurrencyName / CurrencySymbol:** A valuták vizualizációhoz szükséges attribútumai (pl. HUF, Hungarian Forint).

DimCategory: A STGnewsCategory-ből tisztított, kategóriákat tárolja.

- **CategoryID:** Automatikusan generált elsődleges kulcs.
- **CategoryName:** A Python script és az Ollama AI által kigenerált, egyedi kategóriák szótára (pl. Gazdaság, Politika).

DimSource: A RAWnews-ből tisztított, bárhol megjelenő Source adatokat tárolja.

- **SourceID:** Automatikusan generált elsődleges kulcs.
- **SourceName:** A hírügynökségek és portálok egyedi listája (pl. Reuters, CNN), amely alapján vizsgálható a média torzítása vagy fókuszterülete.

DimAuthor: A RAWnews-ből tisztított, bárhol megjelenő szerzői adatokat tárolja.

- **AuthorID:** Automatikusan generált elsődleges kulcs.
- **AuthorName:** A cikkeket publikáló szerzők egyedi listája.

FACT (Tény) Táblák

A ténytáblák tárolják magukat a mérhető adatokat. Tartalmuk nagyrészt a dimenziótáblákra mutató idegen kulcsokból, valamint olyan adatokból áll, amelyeket meg szeretnénk jeleníteni a reportokban/jelentésekben.

FactExchange: A napi devizapiaci mozgások ténytáblája.

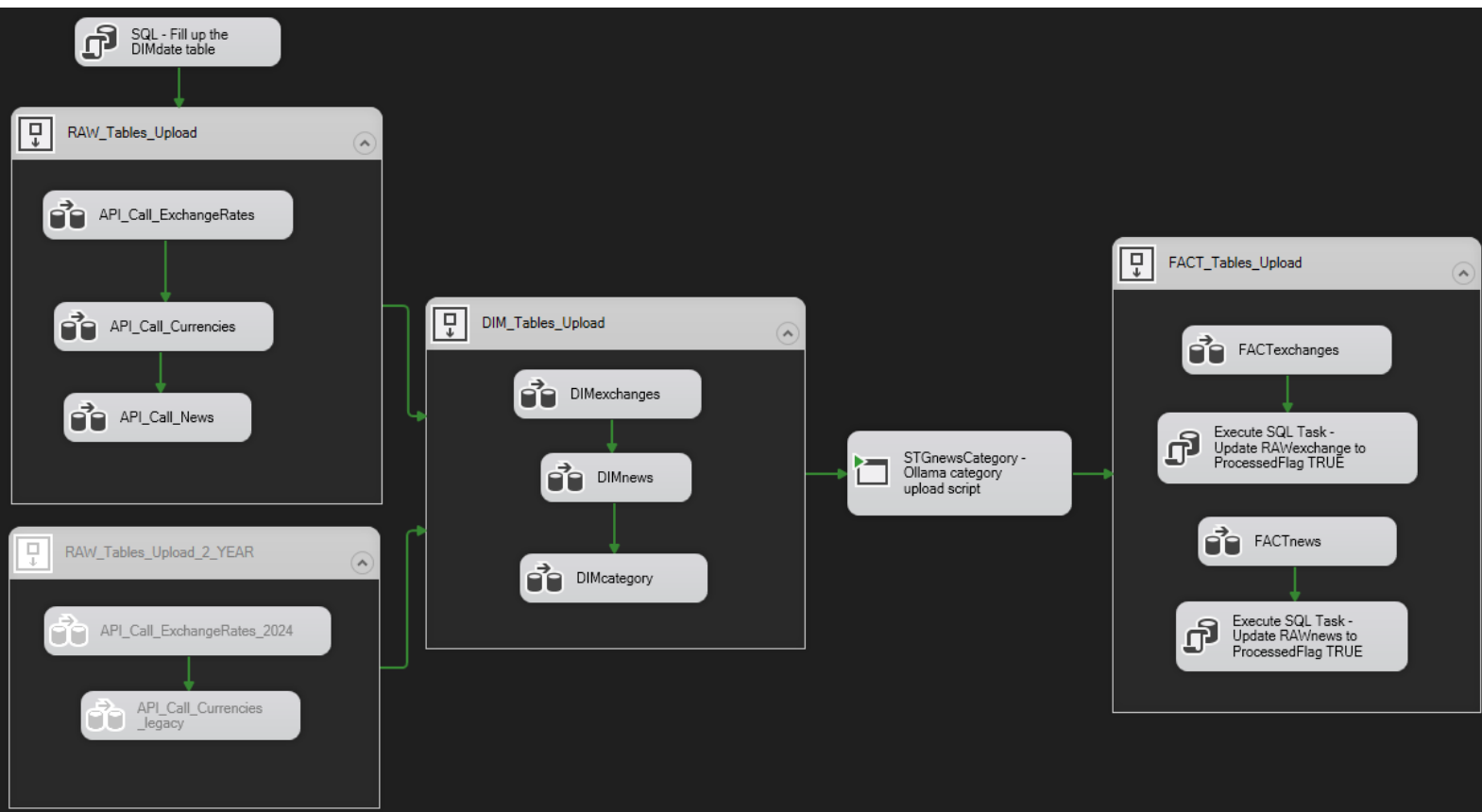
- **ExchangeID:** A tranzakció automatikusan generált elsődleges kulcsa.
- **DateID, BaseCurrencyID, CurrencyID:** Idegen kulcsok a dimenziók felé. Kettős valutakulcsot használunk a viszonyítási alap és a célvaluta megkülönböztetésére.
- **Rate:** A tábla egyetlen klasszikus "tény" adata, a pontos váltási árfolyam.

FactNews: A globális publikációkat rögzítő ténytábla.

- **NewsID:** A hír automatikusan generált elsődleges kulcsa.
- **DateID, SourceID, AuthorID, CategoryID:** Idegen kulcsok a dimenziók felé.
- **Title és Url:** A legfontosabb szöveges adatok, amelyeket a jelentéseken meg akarunk jeleníteni.

ETL jobs, SSIS

Általános folyamatábra



Az SSIS csomag felel a projekt teljes adatintegrációs munkafolyamatának irányításáért. A fejlesztés során a rendszer logikája két fő részre bontható: a **Control Flow**-ra és a **Data Flow**-ra.

A fenti ábrán a projekt átfogó Control Flow-ja látható, amely szigorúan meghatározza a különböző adatfeldolgozási feladatok (ETL jobs) végrehajtási sorrendjét, és a köztük lévő függőségeket. A csomag nagyrészt Data Flow feladatok alkotják és ezeket egészíti ki három Execute SQL Task, valamint egy külső Python szkriptet indító vezérlőelem.

DIMdate tábla feltöltése

Mindenek előtt feltöltjük a DIMdate táblát.

Mivel ez egy olyan tábla, amiben nem csak azokat a dátumokat szeretnénk tárolni, amelyeket a hírek vagy valutaárfolyamok ténylegesen felhasználnak, így a tárolt adatoktól függetlenül tudjuk feltölteni a táblát.

Az itt látható SQL kóddal töltjük fel 2010-től egészen a futtatás évétől számított 2 évig bezáróan a táblát.

```
Enter SQL Query

DECLARE @CurrentDate DATE = '2010-01-01';
DECLARE @EndDate DATE = DATEADD(YEAR, 2, GETDATE());

WHILE @CurrentDate <= @EndDate
BEGIN
    IF NOT EXISTS (SELECT 1 FROM [dbo].[DimDate] WHERE [Date] = @CurrentDate)
    BEGIN
        INSERT INTO [dbo].[DimDate] (
            [Date],
            [Year],
            [Month],
            [Day]
        )
        VALUES (
            @CurrentDate,
            YEAR(@CurrentDate),
            MONTH(@CurrentDate),
            DAY(@CurrentDate)
        );
    END
    SET @CurrentDate = DATEADD(DAY, 1, @CurrentDate);
END
```


A RAW táblák feltöltése

A második lépés a RAW táblák feltöltése.

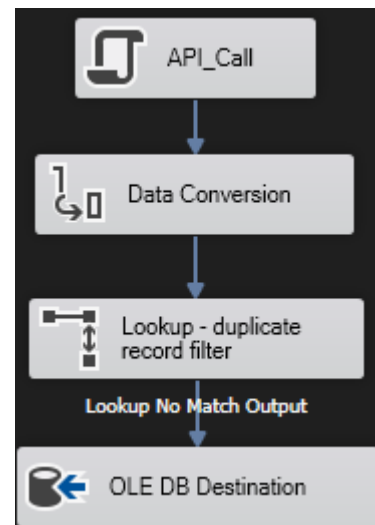
Erre két mód található a Data Flow-ban:

Az egyik az általános, naponta futtatható csomag, amely feladata meghívni az API-kat, feldolgozni a kapott válaszokat, majd feltölteni a RAW táblákat.

A 3 RAW tábla feltöltése közel azonos a Data Flow-ban:

1. C# script component

Ez felelős a megfelelő API hívásáért és feldolgozásáért. A komponensben létrehoztam mindig az API JSON válasznak megfelelő Class-t ami reprezentálja a kapott adatok szerkezetét. Ennek a segítségével tudjuk feldolgozni az API kérést, majd végig iterálni a listán, és a komponens kimenetére tudjuk kötni az adatokat.



```
public override void CreateNewOutputRows()
{
    try {
        string ApiUrl = "https://api.frankfurter.dev/v2/currencies";
        var client = new HttpClient();
        var response = client.GetStringAsync(ApiUrl).Result;

        JavaScriptSerializer serializer = new JavaScriptSerializer();
        List<CurrencyJson> result = serializer.Deserialize<List<CurrencyJson>>(response);

        DateTime apiFetchedAt = DateTime.Now;
        bool processedFlag = false;

        foreach (var currency in result) {
            CurrencyOutputBuffer.AddRow();
            CurrencyOutputBuffer.Enddate = DateTime.Parse(currency.end_date);
            CurrencyOutputBuffer.Startdate = DateTime.Parse(currency.start_date);
            CurrencyOutputBuffer.Isonumeric = currency.iso_numeric;
            CurrencyOutputBuffer.Isocode = currency.iso_code;
            CurrencyOutputBuffer.Name = currency.name;
            CurrencyOutputBuffer.Symbol = currency.symbol;

            CurrencyOutputBuffer.APIFetchedAt = apiFetchedAt;
            CurrencyOutputBuffer.ProcessedFlag = processedFlag;
        }
    } catch (Exception e)
    {
        ComponentMetaData.FireError(0, "Currency rate API call", e.Message, "", 0, out bool cancel);
    }
}
```

Bár a logika hasonló, a RAWnews táblát töltő C# szkript felépítése komplexebb mint a RAWcurrency és RAWexchange esetében. Mivel a releváns pénzügyi és geopolitikai híreket célzott kereséssel gyűjtjük, a kód egy előre definiált, 10 különböző témakört tartalmazó tömbön iterál végig és 10 különböző API kérést dolgoz fel így.

A robusztus adatkinyerés érdekében a szkript több védelmi mechanizmust is tartalmaz:

- A szerver túlterhelésének (és a kitiltásnak) elkerülése végett egy 2 másodperces késleltetést (Thread.Sleep) iktat be a lekérdezések közé.
- Null-koaleszkáló operátorokkal (??) védi a puffert a hiányzó JSON mezőktől.

2. Data Conversion

Az API-ból érkező dátummezőket a betöltési hibák elkerülése végett át kell alakítani az SQL Server által elvárt database date [DT_DBDATE] típusra.

Input Column	Output Alias	Data Type
APIFetchedAt	Copy of APIFetchedAt	database date [DT_DBDATE]
PublishedAt	Copy of PublishedAt	database date [DT_DBDATE]

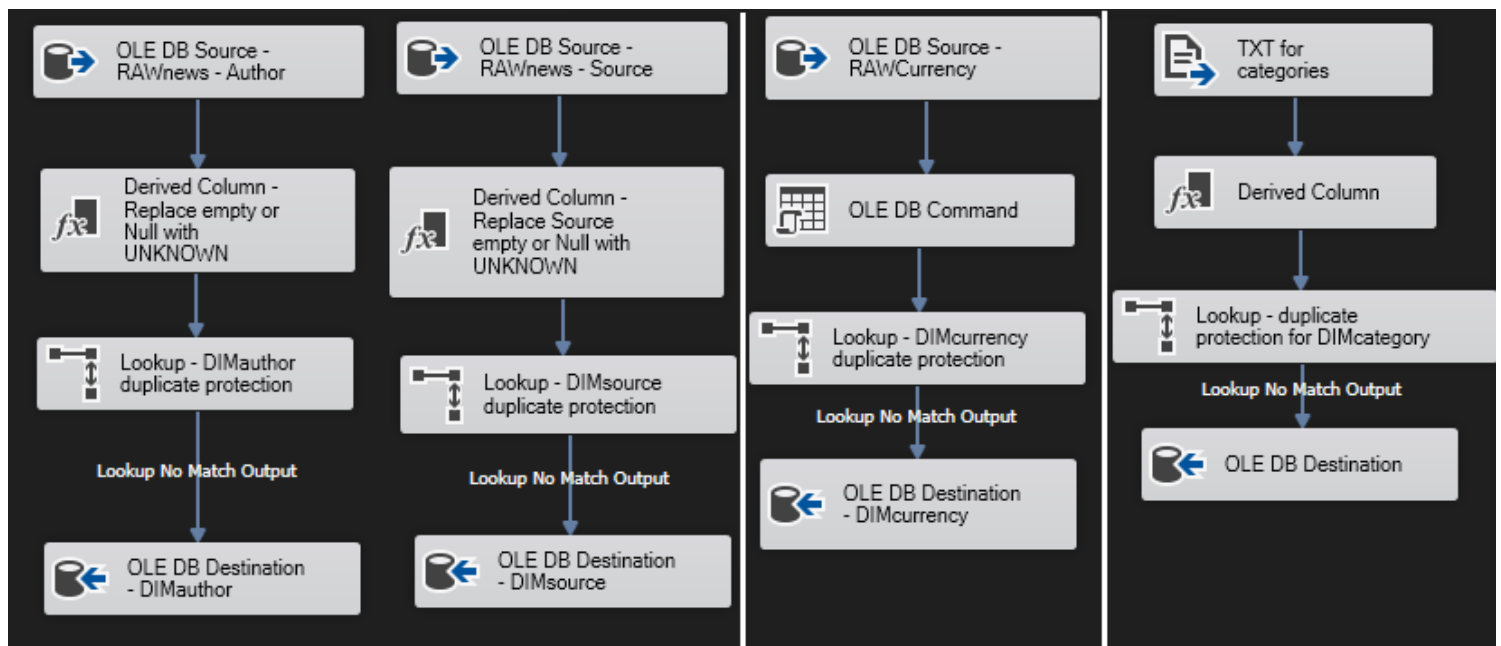
3. Duplikáció védelem

Az adatbázis redundanciájának elkerülése érdekében egy Lookup komponens végzi a szűrést. Ez összehasonlítja a beérkező adatokat a már tárolt rekordok kijelölt attribútumaival. Csak a teljesen új, egyezést nem mutató sorok haladhatnak tovább a betöltés felé a „Lookup No Match Output” ágon.

4. Táblák feltöltése

A duplikációszűrőn sikeresen átjutott, egyedi rekordok végül az OLE DB Destination komponens segítségével fizikailag is beíródnak a cél RAW táblába.

A DIM táblák feltöltése



A harmadik lépés, a dimenzió táblák feltöltése.

A logika közel egységes, és az alábbi lépésekből áll a Data Flow-n belül:

1. Adatkinyerés és Tisztítás.

A feldolgozatlan (ProcessedFlag = 0) nyers adatok beolvasása az adatbázisból. A biztonságos duplikációszűrés érdekében a szöveges mezőket az UPPER() és TRIM() függvényekkel szabványosítjuk. A DIMcategory kivételt képez, hiszen itt a lehetséges kategóriákat egy statikus szöveges fájlból olvassuk be, majd a tisztítást egy Derived Column végzi el.

2. Adattranszformáció.

A DIMauthor és DIMsource ágakon előfordulhatnak hiányzó adatok az attributumokban, így egy Derived Column az üres vagy NULL értékeket alapértelmezett „UNKNOWN” sztringre cseréli.

A DIMcurrency esetében egy OLE DB Command lépés már a folyamat közben 1-re frissíti a feldolgozott rekordok ProcessedFlag értékét a forrástáblában.

3. Duplikáció védelem és adat betöltés.

A korábban már bemutatott Lookup komponens szűri a redundanciát.

A „No Match Output” ágon továbbhaladó, garantáltan új és egyedi rekordok végül az OLE DB Destination segítségével íródnak be a végleges dimenziótáblákba.

Az STG tábla felötlése kategóriákkal

A negyedik lépés, a staging, köztes tábla feltöltése.

A staging, köztes tábla feltöltése egy Python szkript meghívásával történik.

A kategorizáláshoz, a felhőalapú API-k helyett egy lokálisan futtatott, nyelvi modellt alkalmaztam. A Qwen2.5 1.5b verziója ideális választás: hardverigénye alacsony, így lokális gépen is rendkívül gyorsan fut, nulla API költséget generál, és teljes adatbiztonságot garantál.

A szkript lépései:

1. **Dinamikus Inicializálás:** A szkript csatlakozik az SQL Serverhez, és beolvassa a DIMcategory táblából az aktuálisan engedélyezett kategóriák listáját, majd lekérdezi a még feldolgozatlan (ProcessedFlag = 0) híreket a RAWnews táblából.
2. **Determinisztikus Kategorizálás (Few-Shot Prompting):** A szkript végigiterál a cikkeken, és a cím, a leírás, valamint a tartalom alapján kéri a lokális modellt, hogy kategorizálja a megadott kategóriák egyikébe. A modell egy szigorú, példákkal ellátott promptot kap, a kreativitás (Temperature) pedig 0.0-ra van állítva. Ez kényszeríti az AI-t, hogy kizárólag a megadott listából, extra írásjelek nélkül válaszoljon.
3. **Hibakezelés és Betöltés:** Ha a modell a szigorítások ellenére is nem létező kategóriát adna vissza, a szkript egy biztonsági feltétellel automatikusan az „Unknown (Other)” kategóriára cseréli azt. A validált eredmények végül beíródnak a STGnewsCategory táblába.

```
# Ollama hívás qwen2.5:1.5b modellel
response = ollama.chat(
    model='qwen2.5:1.5b',
    messages=[{'role': 'user', 'content': prompt}],
    options={'temperature': 0.0}
)

ai_answer = response['message']['content'].strip()
if ai_answer.endswith('.'): ai_answer = ai_answer[:-1]

final_category = ai_answer if ai_answer in allowed_categories_db else "Unknown (Other)"

results.append((article_id, title, final_category))
```

A FACT táblák feltöltése

Az 5. lépés: A FACT (tény) táblák feltöltése

A folyamat befejező lépése a ténytáblák feltöltése. Célja, hogy a nyers mérőszámokat összekapcsoljuk a DIM és STG táblákból származó azonosítókkal, ID-kkel, ezzel fizikailag is felépítve a relációs csillagsémát.

Data Flow lépései:

1. Adatbázis lekérdezések.

A folyamat továbbra is a feldolgozatlan (ProcessedFlag = 0) rekordokra szűr, de a lekérdezések itt már komplexebb logikát tartalmaznak.

FACTnews kiindulás: Egy INNER JOIN segítségével a nyers híreket már beolvasáskor kibővítjük az STG táblában tárolt, AI által generált kategóriákkal.

```
SELECT DISTINCT r.PublishedAt, r.Source_name, r.Author, r.Title, r.URL, c.GeneratedCategory
FROM RAWnews r INNER JOIN STGnewsCategory c ON r.ArticleID = c.RAWArticleID
WHERE r.ProcessedFlag = 0
```

FACTexchanges kiindulás: Mivel az API optimalizálása miatt csak EUR-bázisú árfolyamokat tárolunk a RAWexchanges-ben, az SQL szkript már a beolvasáskor dinamikusan legenerálja az összes többi keresztárfolyamot.

```
WITH AllRates AS (
    ---1. Az eredeti adatok leszűrése a feldolgozatlan (új) sorokra
    SELECT Date, Currency, Rate
    FROM dbo.RAWexchange
    WHERE ProcessedFlag = 0
    ---
    UNION ALL
    ---
    ---2. Az EUR mesterséges hozzáadása, kizárólag a feldolgozatlan dátumokhoz
    SELECT DISTINCT Date, 'EUR' AS Currency, 1.000000 AS Rate
    FROM dbo.RAWexchange
    WHERE ProcessedFlag = 0
)
---3. A Self-Join a már leszűrt és kibővített listán
SELECT A.Date, A.Currency AS BaseCurrency, B.Currency AS Currency, (B.Rate / A.Rate) AS Rate
FROM AllRates A JOIN AllRates B ON A.Date = B.Date;
```

2. DIM ID-k beszúrása

A hiányzó Source és Author értékeket alapértelmezett „UNKNOWN” sztringre cseréljük, a dátumokat pedig Database Date típusúra konvertáljuk. Ezt követően minden dimenzióhoz egy-egy Lookup komponenst rendelünk, amely kikeresi a szöveges adatokhoz tartozó helyettesítő kulcsot (ID-t) a DIM táblákból, és hozzáadja azt az adatáramhoz.

3. Duplikáció védelem és adat betöltés.

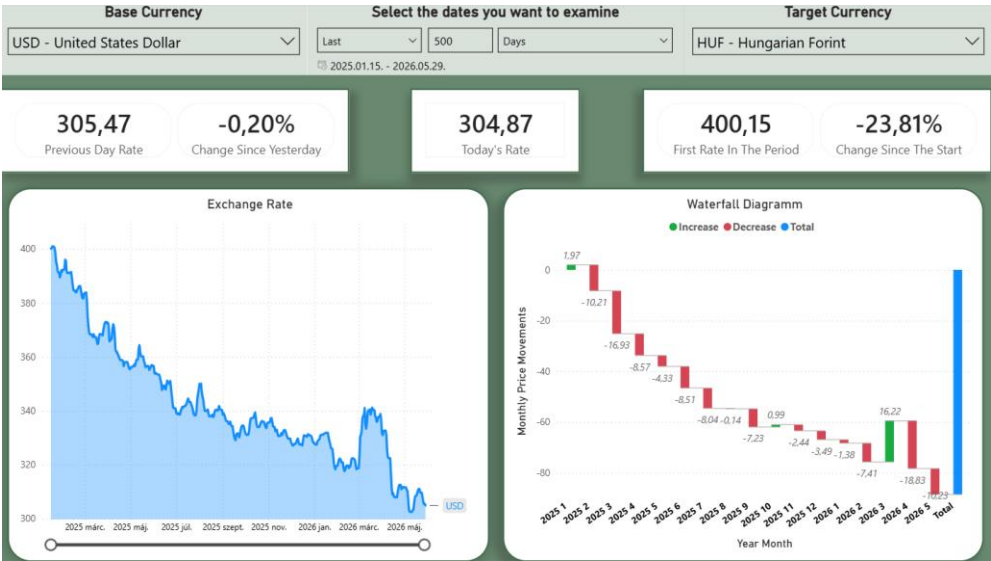
A Data Flow-t egy utolsó Lookup zárja, amely kiszűri a már adatbázisban lévő tény-rekordokat. Az egyedi, új sorok betöltődnek a FACT táblákba, végezetül pedig a rendszer 1-re frissíti a forrásrekordok ProcessedFlag értékét, lezárva az ETL ciklust.

Power BI
View-k

Létrehoztam 2 view-t a valuták jobb elkülönítéséhez a jelentésekben, vw_DimBaseCurrency és vw_DimTargetCurrency.

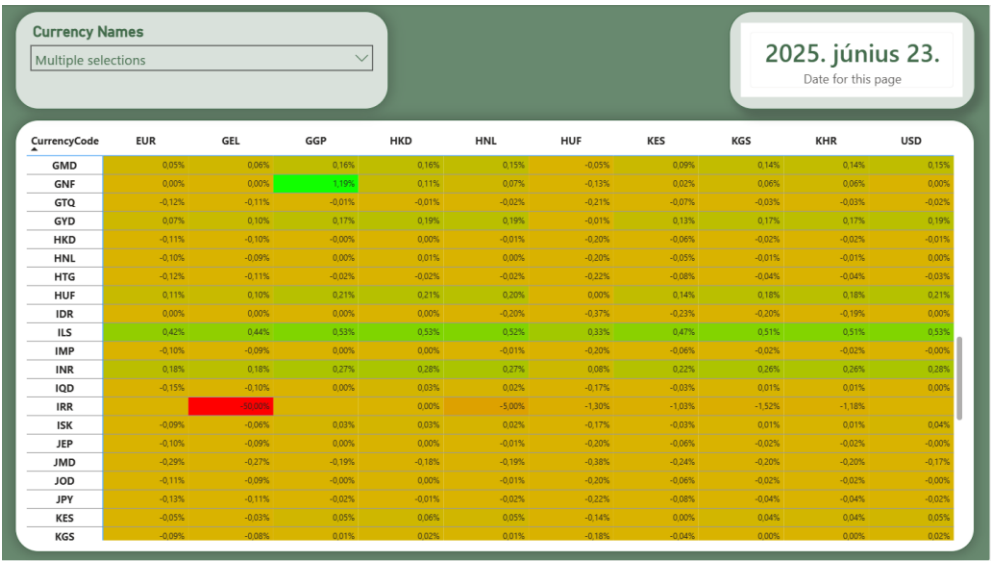
Valuta Árfolyamok

Az oldal egy kiválasztott devizapár elemzését teszi lehetővé egy megadott időszakon belül. A felül található kártyák áttekintést adnak a napi és az időszakos százalékos változásokról. A bal oldali vonaldiagram az árfolyam hosszú távú trendjét, míg a jobb oldali vízesis diagram a havi szintű növekedés és csökkenés mértékét vizualizálja.



Valuta árfolyam mátrix, hőkép

Ez az oldal egy adott nap átfogó piaci pillanatképét mutatja be. A mátrix vizualizáció a kiválasztott devizák előző napi árfolyamhoz viszonyított, napi szintű százalékos elmozdulását ábrázolja. A beépített feltételes formázás zöld, sárga, piros színskálájának köszönhetően az elemzők azonnal, vizuálisan is azonosíthatják a kiugró piaci mozgásokat és az extrém árfolyam-ingadozásokat.



Felhasznált DAX kódok: A DAX (Data Analysis Expressions) a Power BI beépített képletnyelve, amellyel komplex számításokat, aggregációkat és logikai feltételeket hozhatunk létre a relációs adatmodellünkben.

```
Waterfall_Variance =  
VAR CurrentAvg = AVERAGE('FactExchange'[Rate])  
VAR PrevAvg = CALCULATE(AVERAGE('FactExchange'[Rate]), PREVIOUSMONTH('DimDate'[Date]))  
  
RETURN  
CurrentAvg - PrevAvg
```

A vízésés diagram alapjaként kiszámítja a havi nettó árfolyam-elmozdulást úgy, hogy az aktuális hónap átlagából kivonja a PREVIOUSMONTH függvénnyel lekérdezett előző havi átlagot.

Selected_Rate_Chart: A HASONEVALUE függvény használatával biztosítja, hogy a grafikon csak akkor mutasson átlagos árfolyamot, ha pontosan egy alap és egy célvaluta van kiválasztva, ellenkező esetben üres értéket ad vissza, megelőzve ezzel a devizapárok értelmetlen átlagolását.

Latest_Rate: A CALCULATE függvény segítségével az aktuális szűrési kontextus maximális dátumára szűr, így visszaadja a vizsgált időszak legutolsó, legfrissebb napi árfolyamát.

Previous_Rate: Változók segítségével határozza meg a legfrissebb dátumot mínusz egy napot, majd az ALL('DimDate') függvénnyel feloldja a naptárszűrőket, hogy a jelenlegi kontextusból "kilátva" sikeresen lekérje az előző napi árfolyamot.

Daily_Percent_Change: A „Latest_Rate” és a „Previous_Rate” függvényekre építve a DIVIDE függvénnyel kiszámítja az árfolyam napi százalékos elmozdulását, hibakezeléssel védve a hiányzó napi adatok ellen.

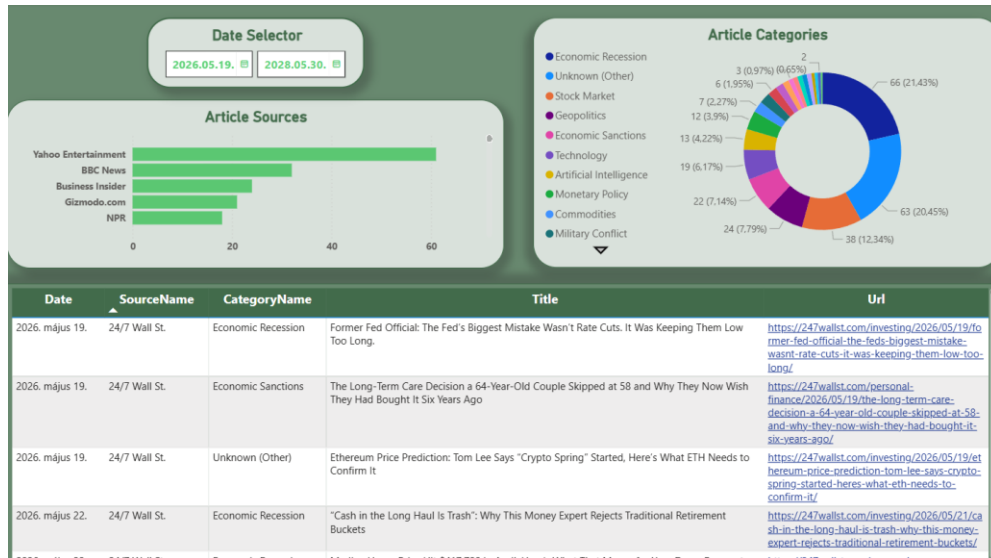
Start_Period_Rate: A „Latest_Rate” mérték logikájának pontos ellentétékeként a MIN függvénnyel a legkorábbi dátumra szűr, visszaadva a szűrt időszak legelső napjának árfolyamát.

Period_Percent_Change: A napi változás matematikai logikáját kiterjesztve, a „Latest_Rate” és a „Start_Period_Rate” mértékek felhasználásával kiszámítja az aktuálisan kiválasztott teljes időkeret, eleje és vége közötti százalékos elmozdulást.

Currency_Label: Szövegösszefűzést végez. A valuta rövid kódját, egy kötőjellel elválasztva, összeköti a teljes névvel.

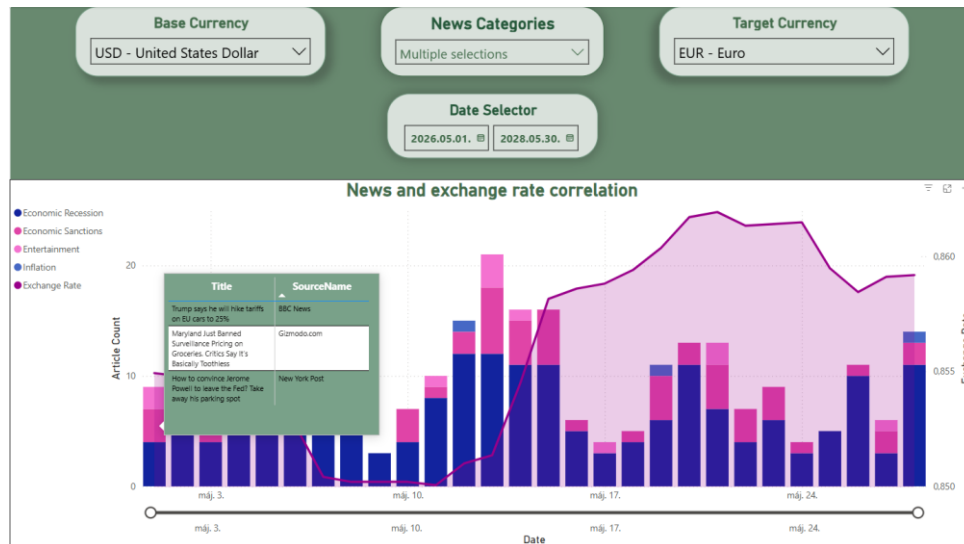
Hírek, média eloszlás

A fáinkdiagram a hírek kategóriánkénti eloszlását, míg a sávdiagram a legaktívabb hírforrásokat vizualizálja. Az alsó, részletes táblázatnak köszönhetően a felhasználók nemcsak számokat látnak, hanem közvetlenül böngészhetnek az adott időszakra szűrt konkrét cikkcímek és az eredeti linkek között is.



Hírek és valuta árfolyam korreláció

Ez a nézet a projekt legfontosabb vizualizációja, amely a közös dátum dimenzióan keresztül egyesíti a két tény táblát. A kombinált diagram egyszerre mutatja a devizaárfolyam mozgását és a napi hírmennyiséget kategóriákra bontva. Egy dinamikus Tooltip is integrálva van benne, hogy az oszlopok fölé húzva az egeret a felhasználó, külön kattintás nélkül is olvashassa az adott naphoz tartozó hírek adatait.



Felhasznált DAX kódok

News_Count: Kiszámolja az aktuálisan leszűrt időszakhoz és kategóriákhoz tartozó hírek teljes darabszámát. A COUNTROWS függvény egyszerűen megszámolja a FactNews tábla sorait, reagálva a felhasználó által beállított szűrőkre.

Fejlesztési tervek

WEB frontend

A jelenlegi Power BI jelentések integrálása egy dedikált, interaktív webes alkalmazásba, Typescript és React alapokon.

AI integration

Text to SQL, korreláció keresés, predikció

A lokális nyelvi modellek (LLM) képességeinek mélyebb kiaknázása az egyszerű kategorizáláson túl.

Egy „Text-to-SQL” funkció bevezetésével a felhasználók kérdéseket tehetnének fel az adatoknak, amit a rendszer automatikusan SQL lekérdezéssé alakít.

Továbbá a kategóriák mellett a cikkek elemzésével az AI nemcsak rejtett korrelációkat tárhatna fel, de a múltbeli mintázatok alapján rövid távú árfolyam-predikciókat is generálhatna.

OLAP réteg

A relációs adattárház és a vizualizációs réteg (Power BI) közé egy SQL Server Analysis Services (SSAS) alapú, OLAP beépítése.

Ahogy az adatbázis gigabájtos méretűre nő, ez a réteg biztosítaná az előre aggregált, azonnali, gyors lekérdezéseket.

A legkomplexebb százalékos és keresztárfolyam-számítások tehermentesíthetők lennének a Power BI memóriájából, és átkerülnének egy dedikált, erre optimalizált szerverkörnyezetbe.

RAW and FACT/DIM táblák külön adatbázisokban

A jelenlegi egyetlen közös adatbázis architektúra felváltása egy fizikailag is elkülönített rendszerre, ahol a nyers, átmeneti adatok és a megtisztított Csillagséma külön SQL adatbázisokban kapnak helyet.